JPA

Day 2

Relationships, Querying & the JPA API

Goal: model real, connected domains correctly, and query them the right way — not just the way that happens to work.

ITI – Java/Enterprise Track

Quick Recap — Day 1

- JDBC gives rows, not objects → ORM bridges that gap
- Hibernate: `Configuration` → `SessionFactory` → `Session` → `Transaction`
- Entity lifecycle: **transient** → **persistent** → **detached** → **removed**
- JPA standardizes Hibernate's ideas: `EntityManagerFactory` / `EntityManager` ≈ `SessionFactory` / `Session`
- Today we connect entities to each other, and learn to query them properly

---

Today's Roadmap

Topic
Entity Associations (`@OneToMany` , `@ManyToOne` , `@OneToOne` , `@ManyToMany`)
Owning side vs. inverse side, `mappedBy`
`FetchType.LAZY` vs `EAGER` , and `LazyInitializationException`
`CascadeType` options
Inheritance mapping strategies
Querying with JPQL/HQL, and a taste of the Criteria API
JPA proper: `EntityManager` , `persistence.xml` , annotations vs. XML
Hands-on lab: Author–Book domain

Entity Associations

What is an Association?

An **association** models a relationship between two entities — exactly like a foreign key models a relationship between two tables, but expressed as an **object reference** instead.

Relationship	Annotation	Real-world example
Many-to-one	`@ManyToOne`	Many `Book`s belong to one `Author`
One-to-many	`@OneToMany`	One `Author` has many `Book`s
One-to-one	`@OneToOne`	One `Employee` has one `ParkingSpace`
Many-to-many	`@ManyToMany`	Many `Student`s enroll in many `Course`s

Unidirectional vs. Bidirectional

- **Unidirectional** — only one side of the relationship knows about the other. E.g. `Book` knows its `Author`, but `Author` has no `books` field at all.
- **Bidirectional** — both sides know about each other. `Book` has an `author` field, **and** `Author` has a `books` field.

> Bidirectional is more convenient to navigate in code (`author.getBooks()`), but requires careful handling to keep both sides in sync — that's exactly what `mappedBy` and owning/inverse sides are about.

Owning Side vs. Inverse Side

In a bidirectional relationship, the database only has **one** foreign key column — so JPA needs to know which Java side is "in charge" of it.

- **Owning side** — the entity that holds the foreign key column, and whose annotation JPA actually uses to generate `INSERT` / `UPDATE` SQL for the relationship
- **Inverse side** — the "read-only" side from the database's point of view; declared using `mappedBy`, pointing back at the field name on the owning side

> **Rule of thumb:** the side with the foreign key (`@ManyToOne`) is almost always the owning side. `@OneToMany` is almost always the inverse side, using `mappedBy`.

`@ManyToOne` / `@OneToMany` — Bidirectional Example

```
@Entity
public class Author {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String name;
```

```java
    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Book> books = new ArrayList<>();

    // getters/setters, plus a helper to keep both sides in sync:
    public void addBook(Book book) {
        books.add(book);
        book.setAuthor(this);
    }
}
```

```java
@Entity
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String title;

    @ManyToOne
    @JoinColumn(name = "author_id")   // this is the actual FK column
    private Author author;

    // getters/setters
}
```

🔍 Line-by-Line: `Author` ↔ `Book`

Code	Meaning
`@ManyToOne` on `Book.author`	`Book` is the **owning side** — many books relate to one author
`@JoinColumn(name = "author_id")`	Explicitly names the foreign key column in the `book` table. Optional — JPA would default to `author_id` anyway, but being explicit is good practice
`@OneToMany(mappedBy = "author", ...)` on `Author.books`	`Author` is the **inverse side** — `mappedBy` points at the `author` field on `Book`, telling JPA "don't create a second join table, the FK already lives on `Book`"
`cascade = CascadeType.ALL`	Operations on `Author` (persist, remove, ...) cascade down to its `books` — covered in detail shortly
`orphanRemoval = true`	If a `Book` is removed from the `books` list, it is automatically deleted from the database, not just unlinked
`addBook(Book book)` helper	A **very common pattern**: with bidirectional relationships, you must update *both* sides yourself — JPA does not do this for you

---

Resulting Database Tables — `Author` ↔ `Book`

```
author                  book
├── id   (PK)           ├── id       (PK)
```

```
├── name                    ├── title
                            └── author_id   (FK → author.id)
```

- **One** foreign key column, `author_id`, living in `book` — because `Book` is the owning side
- `author` has **no** extra column for the relationship at all — the `@OneToMany(mappedBy = "author")` side never generates a column, it only tells JPA where to look
- No join table is created for `@OneToMany` / `@ManyToOne` — that's only needed for `@ManyToMany`, shown later

---

Unidirectional Alternative

If `Author` never needs to be navigated *from* a `Book` collection, skip the inverse side entirely:

```java
@Entity
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String title;

    @ManyToOne
    @JoinColumn(name = "author_id")
    private Author author;

    // no "books" field on Author at all
}
```

- Simpler — one less field to keep in sync
- You lose the ability to write `author.getBooks()` — you'd query for it instead
- **Prefer unidirectional** unless you actually need to navigate from the "one" side

---

Resulting Database Tables — Unidirectional Version

```
author                  book
├── id   (PK)           ├── id         (PK)
├── name                ├── title
                        └── author_id   (FK → author.id)
```

Identical schema to the bidirectional version on the previous slide.

> Unidirectional vs. bidirectional is a **Java-object-model decision only** — it never changes the database schema. The `author_id` column exists either way; the only

> difference is whether `Author` has a `books` field to navigate to it from the Java side.

`@OneToOne` — Unidirectional

```
@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String name;

    @OneToOne
    @JoinColumn(name = "parking_space_id")
    private ParkingSpace parkingSpace;
}

@Entity
public class ParkingSpace {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String location;
    // no back-reference to Employee
}
```

`Employee` holds the foreign key (`parking_space_id`) — `Employee` is the owning side.

Resulting Database Tables — `Employee` ↔ `ParkingSpace`

```
employee                    parking_space
├── id              (PK)    ├── id       (PK)
├── name                    └── location
└── parking_space_id  (FK, UNIQUE → parking_space.id)
```

- The FK column `parking_space_id` lives on `employee`, since `Employee` is the owning side
- A **`UNIQUE`** constraint on that FK column is what actually enforces "one-to-one" at the database level — without it, this would just be a many-to-one relationship that happens to be treated as one-to-one in Java
- `parking_space` gains no extra column at all

`@OneToOne` — Bidirectional

```java
@Entity
public class ParkingSpace {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String location;

    @OneToOne(mappedBy = "parkingSpace")
    private Employee employee;   // inverse side, read-only from DB's perspective
}
```

- `Employee.parkingSpace` stays the owning side (unchanged from before)
- `ParkingSpace.employee` is the inverse side — `mappedBy` points back at `parkingSpace`
- Same owning/inverse rule as `@OneToMany` / `@ManyToOne`: whichever side does **not** have `mappedBy` holds the actual foreign key

---

Resulting Database Tables — Bidirectional Version

```
employee                        parking_space
├── id              (PK)        ├── id        (PK)
├── name                        └── location
└── parking_space_id  (FK, UNIQUE → parking_space.id)
```

Identical schema to the unidirectional version — `mappedBy` on `ParkingSpace.employee` adds a Java field, but zero new columns. The same "schema doesn't change, only navigability does" rule from `@OneToMany` / `@ManyToOne` applies here too.

`@ManyToMany` — Unidirectional

```java
@Entity
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String name;

    @ManyToMany
    @JoinTable(
        name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private Set<Course> courses = new HashSet<>();
}
```

```java
@Entity
public class Course {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String title;
    // no back-reference to Student
}
```

🔍 Line-by-Line: `@ManyToMany` + `@JoinTable`

Code	Meaning
`@ManyToMany`	Neither side has a direct foreign key — a many-to-many relationship needs a **join table** in between
`@JoinTable(name = "student_course", ...)`	Explicitly defines that join table: its name, and its two foreign key columns
`joinColumns`	The column in the join table pointing back to **this** entity (`Student`)
`inverseJoinColumns`	The column in the join table pointing to the **other** entity (`Course`)
`Set<Course>` (not `List`)	**Best practice** for `@ManyToMany` — avoids duplicate-row issues that can occur with `List` under Hibernate's bag semantics

---

Resulting Database Tables — `Student` ↔ `Course`

```
student              course               student_course (join table)
├── id     (PK)      ├── id     (PK)      ├── student_id  (FK → student.id)
├── name             └── title            └── course_id   (FK → course.id)
```

- Neither `student` nor `course` gets a new column — the relationship lives entirely in the **join table**, `student_course`
- The join table's primary key is normally the **combination** of both foreign key columns (a composite key), since one student/course pair should only appear once
- This is the one association type that **always** needs a third table, regardless of direction

---

`@ManyToMany` — Bidirectional

```java
@Entity
public class Course {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
```

```
    private String title;

    @ManyToMany(mappedBy = "courses")
    private Set<Student> students = new HashSet<>();
}
```

- `Student.courses` stays the owning side (it declared the `@JoinTable`)
- `Course.students` is the inverse side, via `mappedBy = "courses"`

Resulting Database Tables — Bidirectional Version

```
student                course                student_course (join table)
├── id     (PK)        ├── id     (PK)       ├── student_id  (FK → student.id)
├── name               └── title             └── course_id   (FK → course.id)
```

Identical three tables to the unidirectional version. `Course` gains a Java-side `students` field for navigation, but not a single new column — reinforcing, for the third time today, that **direction is a Java concern, never a schema concern.**

> **In real projects**, a plain `@ManyToMany` is often replaced by **two** `@OneToMany` / `@ManyToOne` relationships pointing at an explicit join **entity** (e.g. `Enrollment`) — useful the moment the relationship itself needs extra data, like an enrollment date or a grade.

Fetching Strategies

`FetchType.LAZY` vs. `FetchType.EAGER`

Feature	`LAZY`	`EAGER`
When is the related data loaded?	Only when you actually access it	Loaded immediately when the entity is fetched
Performance	Usually better — avoids loading data you may not need	Can cause unnecessary joins or additional queries
Risk	`LazyInitializationException` if accessed after the persistence context is closed	**Fetch creep** — can unintentionally load large object graphs

`LAZY`

The related entity/collection is **not loaded immediately**.

```
@OneToMany(fetch = FetchType.LAZY)
private List<Employee> employees;
```

Hibernate loads the employees only when they are accessed:

```
department.getEmployees();
```

`EAGER`

The related data is **loaded immediately** when the entity is fetched.

```
@ManyToOne(fetch = FetchType.EAGER)
private Department department;
```

Simple Rule

> **Prefer `LAZY` by default**, especially for collections, and explicitly fetch the data you need for a particular use case.

JPA defaults (worth memorizing):

Association	Default fetch type
`@ManyToOne`	`EAGER`
`@OneToOne`	`EAGER`
`@OneToMany`	`LAZY`
`@ManyToMany`	`LAZY`

Setting Fetch Type Explicitly

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "author_id")
private Author author;
```

- Overriding `@ManyToOne` 's default to `LAZY` is a **very common practice** — eager-by-default associations are a frequent source of accidental performance problems in real applications
- `@OneToMany` and `@ManyToMany` are already `LAZY` by default — rarely need overriding

---

The Classic `LazyInitializationException`

```
Author author;
try (EntityManager em = emf.createEntityManager()) {
    author = em.find(Author.class, 1);
} // EntityManager closes here — "author" is now DETACHED

// BOOM: the books collection was never actually loaded,
// and there's no open EntityManager left to load it from
System.out.println(author.getBooks().size());
```

```
org.hibernate.LazyInitializationException:
failed to lazily initialize a collection of role:
com.training.model.Author.books, could not initialize
proxy - no Session
```

Why it happens: a `LAZY` field is really a proxy. Accessing it triggers a fresh database query — but that query needs an **open** `Session` / `EntityManager`, and by then, it's closed.

Avoiding `LazyInitializationException`

- **Access lazy data while the EntityManager is still open** — the simplest fix, when your architecture allows it

---

`CascadeType`

What is Cascading?

Cascading propagates an operation performed on a parent entity down to its related entities automatically — so you don't have to call `persist()` / `remove()` /etc. on each child yourself.

`CascadeType`	**Propagates...**
`PERSIST`	Saving the parent also saves new children
`MERGE`	Merging (updating) the parent also merges children
`REMOVE`	Deleting the parent also deletes its children
`REFRESH`	Refreshing the parent also refreshes children from the DB
`DETACH`	Detaching the parent also detaches children
`ALL`	All of the above

Cascade Example

```java
@OneToMany(mappedBy = "author", cascade = CascadeType.ALL)
private List<Book> books = new ArrayList<>();
```

```java
Author author = new Author();
author.setName("Naguib Mahfouz");

Book book = new Book();
book.setTitle("Palace Walk");
author.addBook(book);   // sets both sides, as shown earlier

em.persist(author);     // book is persisted automatically too — CascadeType.PERSIST
```

- Without `cascade = CascadeType.ALL`, you would need `em.persist(book)` as a **separate**, explicit call
- `orphanRemoval = true` additionally deletes a `Book` the moment it's **removed from the `books` list**, even without deleting the `Author`

> Use cascading only where the child's lifecycle genuinely depends on the parent's (e.g. an `Order`'s `OrderLine`s). Don't cascade `REMOVE` on relationships like `Book` → `Author`, where deleting a book should obviously not delete the author.

Inheritance Mapping Strategies

Why Map Inheritance at All?

Java supports class inheritance; plain relational tables don't have an equivalent concept. JPA offers **three strategies** for representing a Java inheritance hierarchy in the database, plus `@MappedSuperclass` for when you don't need polymorphic queries at all.

```java
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)  // <-- pick one
@DiscriminatorColumn(name = "employee_type")
public abstract class Person {
    @Id @GeneratedValue
    private int id;
    private String name;
}

@Entity
@DiscriminatorValue("EMP")
public class Employee extends Person {
    private double salary;
}

@Entity
@DiscriminatorValue("CUST")
public class Customer extends Person {
```

```
    private String loyaltyLevel;
}
```

Strategy 1 — `SINGLE_TABLE`

One table for the entire hierarchy, with a discriminator column identifying each row's actual subtype.

Pros	Cons
Best query performance — no joins ever needed	Subtype-specific columns must all be nullable
Simplest schema	Table can get wide with many subtypes

```
person
├── id | name | employee_type | salary | loyalty_level
├── 1  | Sara | EMP          | 12000  | NULL
└── 2  | Omar | CUST         | NULL   | Gold
```

Strategy 2 — `JOINED`

One table per class — the subtype table only holds its own columns, joined back to the parent table via a shared primary key.

Pros	Cons
Normalized — no nullable columns, no wasted space	Requires a `JOIN` to reconstruct a full subtype row
Reflects the OO model cleanly	More complex SQL, slightly slower

```
person              employee            customer
├── id | name       ├── id (FK) | salary  ├── id (FK) | loyalty_level
├── 1  | Sara       ├── 1       | 12000
└── 2  | Omar                             ├── 2       | Gold
```

Strategy 3 — `TABLE_PER_CLASS`

One table per concrete class, each containing **all** inherited columns duplicated — no shared parent table at all.

Pros	Cons
Each table is self-contained, fast for single-subtype queries	Polymorphic queries across the whole hierarchy require a `UNION`
No joins needed for a known subtype	Duplicated columns across tables

```
employee                    customer
├── id | name | salary       ├── id | name | loyalty_level
├── 1  | Sara | 12000        ├── 2  | Omar | Gold
```

Choosing a Strategy

Strategy	Best when
`SINGLE_TABLE`	The hierarchy is small, queries need to be fast, and a few nullable columns are acceptable — **the default, and most common in practice**
`JOINED`	You care about a normalized schema more than raw query speed
`TABLE_PER_CLASS`	You rarely query across the whole hierarchy polymorphically

(We'll keep this light today — the goal is recognizing the three strategies and picking sensibly, not mastering every edge case.)

What is JPQL?

JPQL (Jakarta Persistence Query Language) is an **object-oriented** query language — it queries against **entities and their fields**, not tables and columns.

```
TypedQuery<Book> query = em.createQuery(
    "SELECT b FROM Book b WHERE b.title = :title", Book.class);
query.setParameter("title", "Palace Walk");
List<Book> results = query.getResultList();
```

- `Book`, not `book` (the table) — JPQL refers to the **entity class name**
- `b.title`, not a column name — refers to the **Java field**
- `:title` is a **named parameter** — always prefer this over string concatenation (same SQL-injection reasoning as `PreparedStatement` from Day 1)

More JPQL Examples

```
// Positional parameter
em.createQuery("SELECT b FROM Book b WHERE b.id = ?1", Book.class)
  .setParameter(1, 5)
  .getSingleResult();

// Join across the relationship
em.createQuery(
    "SELECT b FROM Book b WHERE b.author.name = :authorName", Book.class)
  .setParameter("authorName", "Naguib Mahfouz")
  .getResultList();

// Aggregate query
em.createQuery(
    "SELECT a.name, COUNT(b) FROM Author a JOIN a.books b GROUP BY a.name",
    Object[].class)
  .getResultList();
```

- JPQL supports `JOIN`, `WHERE`, `GROUP BY`, `ORDER BY`, aggregate functions (`COUNT`, `AVG`, ...) — very close to SQL, but entity-aware

---

A Taste of the Criteria API

The **Criteria API** builds queries **programmatically**, using Java objects instead of a query string — type-safe, and useful when a query's shape needs to change dynamically at runtime.

```
CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Book> cq = cb.createQuery(Book.class);

Root<Book> book = cq.from(Book.class);
cq.select(book).where(cb.equal(book.get("title"), "Palace Walk"));

List<Book> results = em.createQuery(cq).getResultList();
```

Equivalent JPQL, for comparison:

```
em.createQuery("SELECT b FROM Book b WHERE b.title = :title", Book.class)
  .setParameter("title", "Palace Walk")
  .getResultList();
```

- JPQL: shorter, more readable for fixed queries
- Criteria API: more verbose, but compiles with type-checking, and is the right tool when filters/sorting are assembled dynamically from user input

---

🧪 Hands-On Lab 2

Model an Author–Book domain and query it with JPQL.

1. Create `Author` and `Book` entities with a **bidirectional** `@OneToMany` / `@ManyToOne` relationship (as shown earlier), using `cascade = CascadeType.ALL` and `orphanRemoval = true`
2. Configure `persistence.xml` with an H2 in-memory database, listing both entities
3. Write a small `Main` class that:
   - Creates one `Author` with two or three `Book`s, using the `addBook()` helper to keep both sides in sync, and persists it
   - Fetches all books by a given author's name using JPQL
   - Fetches the same data using `JOIN FETCH`, and compares it to a version that triggers `LazyInitializationException` on purpose (by accessing `author.getBooks()` **after** closing the `EntityManager`)
   - Runs one aggregate JPQL query (e.g. count of books per author)
4. **Optional extension:** add a `Publisher` entity with a `@ManyToOne` from `Book`, and a `Category` entity with a `@ManyToMany` to `Book`, to practice a third relationship type

Estimated time: 40–50 minutes

Recap — Day 2

- Associations model relationships as object references: `@OneToMany`, `@ManyToOne`, `@OneToOne`, `@ManyToMany`
- The owning side holds the foreign key; the inverse side uses `mappedBy`
- `LAZY` avoids loading data you don't need — but only works while the `EntityManager` is still open
- `CascadeType` propagates operations from parent to child automatically
- Inheritance in JPA maps to `SINGLE_TABLE`, `JOINED`, or `TABLE_PER_CLASS`
- JPQL queries entities and fields, not tables and columns — HQL is Hibernate's superset of it
- The Criteria API builds the same queries programmatically, for dynamic cases
- Annotations and `orm.xml` produce identical mappings — annotations are the default we'll keep using

---

Coming Up — Day 3

- Spring's role: `LocalContainerEntityManagerFactoryBean`, `PlatformTransactionManager`, `@Transactional`
- Spring Data JPA: `JpaRepository`, derived query methods, `@Query`
- Paging & sorting, auditing basics
- Capstone lab: replace today's manual `EntityManager` code with Spring Data repositories

Questions?